

Projets Pratiques à Réaliser

20 projets concrets classés par niveau pour construire ton portfolio et impressionner les recruteurs — du débutant au niveau avancé

Pourquoi faire des projets personnels ?

Un projet bien documenté sur GitHub vaut plus qu'une ligne de CV. Les recruteurs cherchent des candidats capables de **résoudre des vrais problèmes** avec des vraies données. Chaque projet de ce guide est accompagné d'une description précise, des compétences développées, des apprentissages critiques (AC) IDU correspondants, et d'un niveau de difficulté.

Règle d'or : ne copie jamais un projet de cours (Iris, Titanic). Utilise des données réelles et résous un problème qui t'intéresse vraiment. La motivation se sent dans le code.

■ Niveau 1 — Débutant

■ Niveau 2 — Intermédiaire

■ Niveau 3 — Avancé

Projet #01	Analyse des prix de l'immobilier (DVF)	■ 2-3 semaines
Contexte	Utilise les données DVF (Demandes de Valeurs Foncières) publiées par data.gouv.fr. Analyse l'évolution des prix m² par ville et par année.	
Objectifs	• Nettoyer un jeu de données réel avec Pandas • Réaliser des analyses exploratoires (EDA) • Créer des visualisations avec Matplotlib/Seaborn • Identifier des patterns selon la géographie	
Stack technique	Python, Pandas, Matplotlib, Seaborn, Jupyter Notebook	
Livrable	Notebook Jupyter avec EDA complète + dashboard simple	
■ Bonus	Ajouter une carte choroplèthe avec Folium ou Plotly	
ACs IDU	AC2.2, AC2.6	

Projet #02	Scraping & analyse des offres d'emploi tech	■ 2 semaines
Contexte	Scrape les offres d'emploi sur Indeed ou Welcome to the Jungle pour des métiers data. Analyse les compétences les plus demandées, les salaires proposés et les localisations.	
Objectifs	• Maîtriser BeautifulSoup et Requests • Gérer la pagination et l'anti-scraping • Stocker dans SQLite ou CSV • Visualiser les tendances du marché	
Stack technique	Python, BeautifulSoup, Requests, SQLite, Pandas, WordCloud	
Livrable	Script de scraping + rapport d'analyse des tendances	
■ Bonus	Automatisation hebdomadaire avec GitHub Actions	
ACs IDU	AC2.1, AC2.2, AC2.6	

Projet #03	Dashboard météo avec API publique	■ 2-3 semaines
Contexte	Crée un dashboard interactif qui récupère des données météo depuis l'API OpenWeatherMap et affiche les prévisions et historiques pour plusieurs villes.	
Objectifs	• Consommer une API REST avec authentification • Stocker les données en base (PostgreSQL ou SQLite) • Créer un dashboard avec Streamlit ou Dash • Gérer les erreurs et les retries API	
Stack technique	Python, Requests, PostgreSQL, Streamlit ou Dash	
Livrable	Application web déployée sur Heroku ou Streamlit Cloud	
■ Bonus	Alertes e-mail automatiques si conditions extrêmes	
ACs IDU	AC2.1, AC2.2, AC1.4, AC1.5	

Projet #04	Système de recommandation de films (MovieLens)	■ 3 semaines
Contexte	Utilise le dataset MovieLens (100k ratings) pour construire un système de recommandation basique. Implémente le filtrage collaboratif et le filtrage basé sur le contenu.	
Objectifs	• Comprendre les algorithmes de recommandation • Implémenter KNN et SVD (Surprise library) • Évaluer avec RMSE et MAE • Créer une interface de test simple	
Stack technique	Python, Pandas, Scikit-surprise, Scikit-learn, Streamlit	
Livrable	Système de recommandation + comparaison des algorithmes	
■ Bonus	Déploiement avec FastAPI et interface React simple	
ACs IDU	AC2.5, AC2.10, AC1.5	

Projet #05	Pipeline ETL complet avec Airflow	■ 4-5 semaines
Contexte	Construis un pipeline de données automatisé qui collecte des données financières (Yahoo Finance API), les transforme et les stocke dans un Data Warehouse. Orchestration avec Apache Airflow.	
Objectifs	• Concevoir un pipeline ETL modulaire • Orchestrer avec Airflow (DAGs, tâches, dépendances) • Transformer avec Pandas ou dbt • Charger dans PostgreSQL ou DuckDB	
Stack technique	Python, Apache Airflow, PostgreSQL, dbt, Docker	
Livrable	Pipeline Airflow dockerisé avec DAG documenté	
■ Bonus	Alertes Slack en cas d'échec + monitoring Grafana	
ACs IDU	AC2.8, AC2.9, AC2.13, AC1.9	

Projet #06	Détection de fraude par carte bancaire	■ 4 semaines
Contexte	Utilise le dataset Kaggle 'Credit Card Fraud Detection' pour construire un modèle de classification binaire. Gère le déséquilibre de classes (0.17% de fraudes).	
Objectifs	• Gérer les classes déséquilibrées (SMOTE, class_weight) • Comparer plusieurs algorithmes (RF, XGBoost, LightGBM) • Interpréter les modèles (SHAP values) • Construire une API de scoring avec FastAPI	
Stack technique	Python, Scikit-learn, XGBoost, SHAP, FastAPI, Docker	
Livrable	Modèle + API REST + rapport d'explicabilité SHAP	
■ Bonus	<i>Interface de démonstration + monitoring de la dérive</i>	
ACs IDU	AC2.10, AC2.11, AC1.8	

Projet #07	Analyse de sentiments Twitter/Mastodon	■ 4 semaines
Contexte	Collecte des tweets ou posts sur un sujet d'actualité via API, effectue une analyse de sentiment, et construit un tableau de bord en temps quasi-réel.	
Objectifs	• Utiliser une API de réseau social avec OAuth • NLP de base : tokenisation, lemmatisation • Modèle de sentiment (VADER, BERT pré-entraîné) • Dashboard temps réel avec Streamlit	
Stack technique	Python, Tweepy/Mastodon.py, VADER ou HuggingFace, Streamlit, SQLite	
Livrable	Dashboard live avec évolution du sentiment sur une semaine	
■ Bonus	<i>Comparaison entre plusieurs sujets/événements</i>	
ACs IDU	AC2.1, AC2.5, AC2.10, AC2.11	

Projet #08	Data Warehouse avec modélisation en étoile	■ 5 semaines
Contexte	Conçois un Data Warehouse complet pour une boutique e-commerce fictive. Modélisation dimensionnelle (étoile/flocon), population avec données synthétiques, requêtes analytiques.	
Objectifs	• Modéliser tables de faits et dimensions • Implémenter les SCD (Slowly Changing Dimensions) • Écrire des requêtes analytiques complexes • Connecter Power BI ou Metabase	
Stack technique	PostgreSQL ou DuckDB, dbt, Python (Faker), Power BI ou Metabase	
Livrable	Schéma DWH + transformations dbt + rapport Power BI	
■ Bonus	Automatisation du chargement incrémental	
ACs IDU	AC1.1, AC2.2, AC2.12, AC2.8	

Projet #09	Application de prédiction de churn télécom	■ 5 semaines
Contexte	Un opérateur télécom veut prédire quels clients vont résilier leur abonnement. Utilise le dataset Telco Customer Churn (Kaggle). Déploie le modèle en application web.	
Objectifs	• Feature engineering sur données clients • Modèle de classification avec cross-validation • Interface utilisateur pour saisie manuelle • Déploiement sur cloud (Heroku/Render)	
Stack technique	Python, Scikit-learn, FastAPI, React ou Streamlit, Docker, Heroku	
Livrable	Application web déployée accessible publiquement	
■ Bonus	A/B testing de deux modèles en production	
ACs IDU	AC2.10, AC1.5, AC1.4, AC3.2	

Projet #10	Base de données NoSQL : Graphe de connaissances	■ 4 semaines
Contexte	Construis un graphe de connaissances avec Neo4j à partir de données Wikipedia (ou IMDb). Implémente des requêtes de recommandation et de recherche de chemins.	
Objectifs	• Modéliser des relations complexes en graphe • Langage de requête Cypher • Algorithmes de graphe (PageRank, communautés) • API GraphQL ou REST pour exposer le graphe	
Stack technique	Neo4j, Python (py2neo), FastAPI, React	
Livrable	Graphe Neo4j + API de requêtage + visualisation	
■ Bonus	Interface de visualisation interactive avec D3.js	
ACs IDU	AC2.3, AC2.12, AC3.2	

Projet #11	Pipeline Big Data avec Spark sur Kubernetes	■ 6-8 semaines
Contexte	Déploie un cluster Spark sur Kubernetes (ou localement avec Kind) et traite un dataset volumineux (> 10 Go). Implémente des transformations complexes et des agrégations distribuées.	
Objectifs	• Déployer Spark en mode standalone ou sur K8s • Écrire des jobs PySpark optimisés • Gérer les partitions et le shuffle • Stocker les résultats au format Parquet ou Delta Lake	
Stack technique	Apache Spark, PySpark, Kubernetes, MinIO (S3-compatible), Docker	
Livrable	Jobs Spark documentés + benchmark de performances	
■ Bonus	Intégration dans un pipeline Airflow complet	
ACs IDU	AC1.11, AC2.13, AC2.14, AC1.14	

Projet #12	Plateforme de monitoring IoT temps réel	■ 8 semaines
Contexte	Simule des capteurs IoT qui envoient des données en continu (température, humidité, vibrations). Traitement en streaming avec Kafka + Flink ou Spark Streaming, alertes et dashboard temps réel.	
Objectifs	• Architecture Lambda ou Kappa pour le streaming • Kafka pour le bus de messages • Traitement en temps réel avec Spark Streaming ou Flink • Dashboard InfluxDB + Grafana	
Stack technique	Python, Apache Kafka, Spark Streaming, InfluxDB, Grafana, Docker Compose	
Livrable	Plateforme IoT complète dockerisée avec 5+ capteurs simulés	
■ Bonus	ML en temps réel pour détection d'anomalies	
ACs IDU	AC1.11, AC1.14, AC2.13, AC2.14	

Projet #13	Moteur de recherche vectorielle RAG	■ 6 semaines
Contexte	Construis un système RAG (Retrieval Augmented Generation) qui permet d'interroger une base documentaire en langage naturel. Utilise ChromaDB ou Pinecone + un LLM (Ollama local ou OpenAI).	
Objectifs	• Comprendre les embeddings et la recherche sémantique • Chunking et indexation de documents PDF/texte • Intégration LLM pour génération de réponses • Interface de chat avec historique	
Stack technique	Python, LangChain ou LlamaIndex, ChromaDB, Ollama, Streamlit	
Livrable	Chatbot documentaire avec interface et évaluation des réponses	
■ Bonus	Évaluation automatique avec RAGAS	
ACs IDU	AC2.16, AC2.10, AC3.2, AC1.5	

Projet #14	MLOps : CI/CD pour un modèle de Machine Learning	■ 8 semaines
Contexte	Mets en place un pipeline MLOps complet : entraînement automatisé, versioning des modèles, tests de régression, déploiement en staging puis production avec rollback automatique.	
Objectifs	• Versionner données et modèles avec DVC • CI/CD avec GitHub Actions • Registre de modèles avec MLflow • Déploiement containerisé avec tests A/B	
Stack technique	Python, MLflow, DVC, GitHub Actions, Docker, FastAPI, Grafana	
Livrable	Pipeline MLOps complet avec documentation de l'architecture	
■ Bonus	Monitoring de la dérive (data drift) avec Evidently	
ACs IDU	AC1.9, AC1.13, AC1.15, AC2.10	

Projet #15	Plateforme Data Lakehouse (Architecture Medallion)	■ 10-12 semaines
Contexte	Construis une architecture Data Lakehouse complète avec les couches Bronze/Silver/Gold. Ingestion depuis plusieurs sources, transformations progressives, exposition via API ou BI.	
Objectifs	• Architecture Medallion (Bronze/Silver/Gold) • Delta Lake ou Apache Iceberg pour les tables • Orchestration Airflow pour les pipelines • Gouvernance et qualité des données (Great Expectations)	
Stack technique	Apache Spark, Delta Lake, MinIO, Apache Airflow, Trino, dbt, Docker	
Livrable	Plateforme complète avec documentation d'architecture + guide d'exploitation	
■ Bonus	Intégration d'un catalogue de données (Apache Atlas ou DataHub)	
ACs IDU	AC1.11, AC1.12, AC2.13, AC2.17	

Comment présenter ses projets sur GitHub

Un projet bien présenté sur GitHub est aussi important que le projet lui-même. Voici la structure recommandée :

Élément	Ce qu'il faut inclure	Pourquoi c'est important
README.md	Titre accrocheur, problème résolu, démo GIF/vidéo, instructions d'installation	C'est la première chose qu'un recruteur voit
requirements.txt ou pyproject.toml	Toutes les dépendances avec versions fixées	Reproductibilité du projet
Structure de dossiers	src/, data/, notebooks/, tests/, docs/	Montre une approche professionnelle
Notebooks Jupyter	Explications Markdown détaillées, visualisations propres	Raconte l'histoire des données
Tests unitaires	pytest pour les fonctions clés	Démarche qualité — appréciée des seniors
CI/CD (GitHub Actions)	Lint (flake8), tests auto à chaque push	MLOps/DevOps mindset
Licence	MIT, Apache 2.0	Professionnel et open source friendly
Données en cache	Ne pas committer les données volumineuses (dvc ou git lfs)	Éviter les fichiers de 500 Mo sur GitHub

Plateformes pour exposer ses travaux

GitHub	Hébergement de code + portfolio projets	github.com
Kaggle	Compétitions + notebooks publics	kaggle.com
Streamlit Community Cloud	Déploiement gratuit d'apps data	streamlit.io
Hugging Face Spaces	Déploiement démos IA/ML gratuit	huggingface.co
DataSciencePortfolio	Site portfolio spécialisé data science	datascienceportfol.io
Medium / Substack	Articles techniques + visibilité	medium.com

Sources : KDnuggets Jan 2025 · InterviewMaster 2025 · Kaggle Datasets · GitHub Topics data-science-projects · Référentiel IDU RNCP35994.